

Academic Year	Module	Assessment Number	Assessment Type
2026	Artificial Intelligence and Machine Learning(6CS012)	2	Report

Traffic sign Image Classification

Student Id: 2431235

Student Name: Sujal Shrestha

University Email: s.shrestha202@wlv.ac.uk

Section: L6CG19

Tutor: Bijaya Ghimire

Abstract

In this project, the implementation and evaluation of the deep learning models to perform image classification of the traffic signs are presented by computer vision methods. The objective of the project is to classify the image of the traffic signs into some categories by using Convolutional neural Networks (CNNs) and transfer learning approach. Testing and experimenting with different architectures such as Baseline CNN, Deeper CNN, comparison of various optimizers, ablation study, and transfer learning from MobileNetV2. To enhance model generalisation and model classification capabilities, data processing methods (data preprocessing, augmentation, Batch Normalisation), as well as Dropout regularisation and transfer learning strategies were used. The models were evaluated based on accuracy, precision, recall, $f(1)_{\text{score}}$ analysis, confusion matrix and classification report. Experimental results proved that the proposed transfer learning technique with MobileNetV2 model achieved best performance as compared to CNN models, which were trained from scratch.

Table of Contents

1.Introduction.....	3
2.Dataset Description and Preparation.....	3
3.Model Architectures.....	4
4.Training Strategy and Evaluation	5
6. Sample Prediction Visualisation	6
7. Overfitting Analysis.....	6
8. Discussion and Critical Analysis	7
10. Conclusion	7
11.References.....	8

1.Introduction

Traffic sign recognition is considered as one of the most crucial fields of computer vision and deep learning, as it plays a vital role in the field of autonomous driving systems and intelligent transportation technologies. Automatic traffic sign classification systems assist vehicles to interpret the information on various traffic signs and indications as well as warn them of the speed limits, while the road is being travelled.

Conventionally image processing is very unreliable and relies heavily on manual feature extraction. In contrast, Convolutional Neural Networks (CNNs) learn directly from the pixel information, and can thus extract features of the images automatically. The CNN-based approaches have been shown to have performed well for image classification due to their ability to represent the spatial relationship and visual characteristics at different hierarchical levels.

In this project, we will investigate multiple CNN approaches to traffic sign image classification. The notebook includes a Baseline CNN model, a CNN with Batch Normalisation and Dropout model, an ablation study, a comparison of optimisers and Transfer Learning from a model pre-trained on ImageNet with MobileNetV2.

The objective of the project will be to evaluate the models based on different metric and to compare the performance of the regularisation technique, augmentation and transfer learning with the aim of improving the ability of the image classification.

2.Dataset Description and Preparation

2.1Dataset Overview

This project will make use of the traffic sign dataset, which is composed of images of traffic signs organised in folders with classes indicating their category. These pictures depict all types of traffic signs connected to caution signs, instruction signs, no-passing signs and speed related traffic signs.

This data has been compressed in zip files, and was extracted during runtime. Pre Training, all the image files have been validated to remove corrupted and unsupported image files using Python Imaging Library (PIL).

Three groups were created from the data:

- 80% training data
- 20% validation data

To ensure balancing classes and within both subsets stratified splitting was used. Experiments were conducted using a fixed value, the random “seed” which is used to make the random values.

2.2Preprocessing and Data Augmentation

All the images were resized to the same resolution prior to training. The intensity of the pixels was divided by 255 to normalise them to lie between [0, 1].

Various techniques were used to mitigate over-fitting; these techniques have been referred to as Data Augmentation techniques. The augmentation pipeline was comprised of:

- Rotation
- Width shifting
- Height shifting
- Zooming
- Brightness variation
- Shearing

Until recently, only rescaling was applied to validation and testing generators; the evaluation is supposed to be done on unmodified images. In case of transfer learning with MobileNetV2 we used an ImageNet-specific set of preprocessing functions as a pre-processing pipeline.

3. Model Architectures

3.1 Baseline CNN

The reference architecture of the project was the simplest, which was called the Baseline CNN. The model consists of a series of convolutional layers, followed by a series of pooling layers, and then fully connected dense layers. The architecture included:

To use the ReLU active, the flag 'relu' can be used to specify the activation layer.

- MaxPooling layers
- Flatten layer
- Dense layers
- Dropout regularisation
- Softmax output layer

Baseline CNN gave satisfactory performance in simple image pattern learning, but failed to give satisfactory performance in learning more complex image pattern.

3.2 Deeper CNN and Regularisation

The second architecture was based on a deeper CNN structure of which there was also regularisation by Batch Normalisation and Dropout. For minimising the internal covariate shift, Batch Normalisation is made with the training to be more stable and converged. The dropout regularization was applied to ensure that randomly de-activated neurons for training to improve the generalisation and anti-over fitting properties. CNN was made deeper than the Baseline CNN, significantly improving feature extraction capability, to learn more complex hierarchical visual features.

3.3 Optimizer Comparison: Adam vs SGD

The education behavior and classification have been contrasted using the deeper representation of CNN trained with both Adam and SGD optimizers. Adam had a quick convergence and manual tuning was significantly cut in this approach due to adaptive learning rates per each parameter. When using optimisation behaviour with momentum, the technique of SGD was smoother, but took more number of epochs to optimise well. Comparing the effect of the optimisation strategy in CNN training was highlighted by using the so-called optimiser comparison.

3.4 Ablation Study: Effect of Dropout

An ablation study was conducted to investigate the importance of Dropout regularisation in CNN structure. Another model was trained with a deeper CNN network (from which all the

Dropout layers were dropped) and the other parameters were as described above. For the no-dropout model, the training accuracy was higher and the validation accuracy was less subjected to changes as compared to the others, leading to a better fit for the model. This procedure revealed that Dropout regularisation was essential for the success of generalisation of the model.

3.5 Transfer Learning with MobileNetV2

Transfer learning was done with a model pre-trained from the ImageNet database using MobileNetV2. The idea of transfer learning is to use a previously learned visual feature representation (the pretrained model) for a new classification task.

Only the parameters of the classification head, trained, were changed and the parameters of the convolution part were kept fixed, which is the part of the MobileNetV2 model. Selected layers were not frozen, but were fine-tuned using a small learning rate after initial training.

The classical head contained the following:

- GlobalAveragePooling2D
- Dropout layer
- Dense layers
- Softmax output layer

Among the models that were tested the best performance was done by overall transfer learning.

4. Training Strategy and Evaluation

4.1 Training Configuration

The loss function to train all models used categorical cross-entropy loss and the metric for all models used to evaluate the performance of the model was the accuracy measure.

A number of training calls were put in place:

- EarlyStopping
- ReduceLROnPlateau
- ModelCheckpoint

In the case where the performance during validation did not improve, it was necessary to stop training, but fortunately, this was not in vain, due to EarlyStopping. The learning rates were automatically decreased when they failed to decrease by ReduceLROnPlateau and the best-performing model weights were saved by ModelCheckpoint.

4.2 Evaluation Configuration

Following that, the subsequent evaluation metrics were picked up:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

- Classification Report

The data set was also found to be class-imbalanced, which was addressed using a data set with so-weighted averages.

Confusion matrices were formed and analysed the classification behavior at the class level for the most frequently misclassified classes, that is traffic signs.

5. Experiment Results and Evaluation

Although Baseline CNN gave acceptable performance on classification tasks, there were limitations in dealing with complex traffic signs due to its simple structure. Deeper CNN structure helped in achieving good performance for the classifier because Batch Normalisation and Dropout were helpful in making training stable and improving generalization, respectively. Adam optimizer was seen to be better performing than SGD in terms of faster convergence. Transfer Learning using MobileNetV2 produced the best results because ImageNet feature representations helped in obtaining good visual understanding before fine-tuning. Evaluation was done using the following criteria:

- Analysis of accuracy
- Comparison of precision and recall scores
- Evaluation of F1 scores
- Analysis of confusion matrices
- Generation of classification reports

Overall, experiment results proved that transfer learning and regularization techniques greatly enhance performance on traffic sign classification tasks.

6. Sample Prediction Visualisation

The notebook had prediction visualization capabilities showing:

- Traffic signs images
- The true label
- The predicted label
- Prediction accuracy score

Corrections were clearly marked differently from errors. The errors made by the classifier mostly arose between visually alike traffic signs.

7. Overfitting Analysis

One of the major issues that can be encountered in CNN-based traffic sign image classification is overfitting, where the model tends to memorize the training set rather than learn generalizable image features. Various regularizations were employed during the training phase to minimize overfitting and maximize the generalization capabilities of the models.

For instance, dropout regularization was included in the CNN models to help randomize which neurons are trained during each epoch. Batch normalization was also incorporated into the training algorithm to stabilize the training process and expedite convergence. Data augmentation procedures like rotating, scaling, shifting, and adjusting the brightness of images were employed to enhance the diversity of the training sets and increase the model's capability to generalize on unseen images.

The Early Stopping and ReduceLROnPlateau callbacks were used to stop unnecessary training once the validation score plateaued to avoid overfitting and ensure consistent validation accuracy.

8. Discussion and Critical Analysis

Firstly, the project shows that CNNs with appropriate regularisation can greatly improve the accuracy of traffic sign image classification. Batch Normalisation made the model converge faster, while Dropout managed to combat overfitting. Adam versus SGD gave insights on optimisation choices, with Adam being able to train faster but SGD being sometimes more stable.

However, the greatest insight is that transfer learning provides a clear advantage in accuracy when compared to learning from scratch. Nevertheless, there are still some limitations in this project, namely:

- No implementation of cross-validation
- No use of Grad-CAM for visualization
- Using Efficient Net for better performance
- No ensembles for improved accuracy

Some ideas for future improvement are:

- Use of transformer architecture for computer vision
- Efficient Net based transfer learning
- Test-time augmentation for better results
- Cross-validation testing
- Optimizations for real-time deployment

10. Conclusion

The present report outlined an elaborate investigation of traffic sign image classification by means of Convolutional Neural Networks (CNNs) and transfer learning. Several different models, namely Baseline CNN, Deeper CNN, Optimiser Comparison, Ablation Study, and MobileNetV2 Transfer Learning, were explored within the same experimental methodology. The main findings include the following facts:

- Architectural depth increases the effectiveness of feature learning
- Batch Normalisation contributes to faster convergence
- Dropout dramatically enhances generalisation
- Adam converges much faster than SGD
- Transfer learning far surpasses training from scratch

In addition, the project emphasised the significance of data augmentation, regularisation, learning rate control, and model validation during training deep neural networks. Overall, the current project delivers a robust framework for traffic sign image classification and highlights the efficiency of computer vision techniques based on CNNs.

11. References

1. LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *Proceedings of the International Conference on Machine Learning (ICML)*.
- Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations (ICLR)*.
- Chollet, F. et al. (2015). Keras. <https://keras.io> TensorFlow Developers. (2024). TensorFlow. <https://www.tensorflow.org>